C Language Operators

1. Introduction to Operators

In **C programming**, an **operator** is a symbol used to perform an operation on variables or values.

The values on which operators act are called **operands**.

Example

```
int a = 10;

int b = 5;

int sum = a + b;
```

In this example:

Element	Meaning
+	Operator
a and b	Operands
a + b	Expression

Therefore:

Operator + Operand(s) = Expression

2. Types of Operators in C

C language provides several categories of operators:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Increment and Decrement Operators
6. Bitwise Operators
7. Conditional (Ternary) Operator
8. Special Operators

3. Arithmetic Operators

Arithmetic operators are used to perform **mathematical calculations**.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b

Example Program

```
#include <stdio.h>

int main()

{

   int a = 20;

   int b = 6;


   printf("Addition = %d\n", a + b);

   printf("Subtraction = %d\n", a - b);

   printf("Multiplication = %d\n", a * b);

   printf("Division = %d\n", a / b);

   printf("Modulus = %d\n", a % b);


   return 0;

}
```

Output

```
Addition = 26
```

Subtraction = 14

Multiplication = 120

Division = 3

Modulus = 2

Important Note

- / performs **integer division** if both operands are integers.
- % only works with **integers**.

Example:

7 / 2 = 3

7 % 2 = 1

4. Relational Operators

Relational operators are used to **compare two values**.

The result is either:

- **1 (true)**
- **0 (false)**

Operator	**Meaning**
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example

```
#include <stdio.h>


int main()
{
    int a = 10;
    int b = 20;


    printf("%d\n", a == b);
    printf("%d\n", a != b);
    printf("%d\n", a > b);
    printf("%d\n", a < b);


    return 0;
}
```

Output

```
0
1
0
1
```

Explanation:

- a == b → false → **0**
- a != b → true → **1**

5. Logical Operators

Logical operators are used to **combine multiple conditions**.

They are commonly used in **decision-making statements** such as if, while, and for.

Operator	Meaning
&&	Logical AND
`	
!	Logical NOT

Logical AND (&&)

Returns **true only if both conditions are true**.

Example:

if(age >= 18 && age <= 60)

Both conditions must be true.

Logical OR (||)

Returns **true if at least one condition is true**.

Example:

if(day == 6 || day == 7)

True if day is **Saturday OR Sunday**.

Logical NOT (!)

Reverses the condition.

Example:

if(!(a > b))

Means **NOT (a > b)**.

6. Assignment Operators

Assignment operators are used to **assign values to variables**.

Operator	Meaning	Example
=	Assign	a = 10
+=	Add and assign	a += 5
-=	Subtract and assign	a -= 3
*=	Multiply and assign	a *= 2
/=	Divide and assign	a /= 4
%=	Modulus and assign	a %= 2

Example

```
int a = 10;


a += 5;   // a = a + 5
```

New value of a becomes **15**.

7. Increment and Decrement Operators

These operators increase or decrease a variable by **1**.

Operator	Meaning
++	Increment
--	Decrement

Increment Example

```
int a = 5;
a++;
```

Now:

a = 6

Decrement Example

int a = 5;

a--;

Now:

a = 4

8. Prefix vs Postfix

Increment and decrement can appear **before or after the variable**.

Type	Example
Prefix	++a
Postfix	a++

Example

int a = 5;

int b;

b = a++;

Result:

b = 5

a = 6

Explanation:

- First use a
- Then increment it.

Prefix Example

```
int a = 5;

int b;


b = ++a;
```

Result:

a = 6

b = 6

Explanation:

- First increment
- Then use the value.

9. Conditional (Ternary) Operator

The ternary operator is a **short form of the if-else statement**.

Syntax

```
condition ? expression1 : expression2;
```

Meaning:

- If condition is **true → expression1**
- If condition is **false → expression2**

Example

```
int a = 10;

int b = 20;

int max = (a > b) ? a : b;

printf("Maximum = %d", max);
```

Output

Maximum = 20

10. Bitwise Operators

Bitwise operators work on **binary numbers (bits)**.

Operator	Meaning
&	Bitwise AND
`	`
^	Bitwise XOR
~	Bitwise NOT
<<	Left Shift
>>	Right Shift

Example:

```
int a = 5;   // 101

int b = 3;   // 011

printf("%d", a & b);
```

Result:

1

11. Operator Precedence

Operator precedence determines **the order in which operations are performed**.

Example:

```
int result = 10 + 5 * 2;
```

Multiplication happens first.

5 * 2 = 10

10 + 10 = 20

Result:

20

Simplified Precedence

Priority	Operators
High	++ --
	* / %
	+ -
	< <= > >=
	== !=
	&&
	`
Low	=